FINAL PROJECT ASSIGNMENT

Integrated Dependability Engineering Capstone

Course: Fault Tolerance and Dependable Computing

1. Project Overview

The Final Project is an integrated engineering assessment in which teams design, implement, and evaluate a dependable distributed system. Teams may continue with the university information system used in the midterm/endterm assignments or propose another instructor-approved system with comparable complexity, such as a scholarship-processing platform, online booking service, inventory system, or sensor-data service.

The project must combine system-level dependability analysis, reliability engineering, infrastructure and software fault tolerance, microservices resilience, monitoring/observability, security, and systematic verification. The emphasis is on a defensible engineering process and measured evidence, not merely on feature count.

2. Learning Objectives

- Translate service requirements into measurable dependability, availability, recovery, and data-integrity objectives.
- Construct a fault model and analyze critical failure paths and single points of failure.
- Design and implement complementary infrastructure-level and software-level fault-tolerance mechanisms.
- Instrument the system to detect, investigate, and communicate operational incidents.
- Apply security controls and evaluate the system under selected fault, load, and misuse scenarios.
- Use repeatable verification, validation, and fault-injection experiments to assess whether objectives are met.
- Communicate trade-offs, limitations, and recommendations using technical evidence.

3. Project Selection and System Scope

Select one system with at least three independently identifiable services/modules, one persistent data store, and at least one critical workflow involving multiple steps or components. Define the users, main workflow, assumptions, data handled, and consequences of failure.

- Option A: continue the university information system from the midterm and endterm projects.
- Option B: propose another distributed system and obtain instructor approval before implementation.
- The scope must be small enough to implement and test within the course but complex enough to demonstrate fault containment, recovery, and measurable trade-offs.

4. Required Engineering Work

Work package	Minimum requirements	Evidence
A. Requirements and dependability model	Define critical functions, service-level objectives (SLOs), assumptions, fault/error/failure terminology, and at least five failure scenarios.	Requirements table, fault model, and failure-impact analysis.
B. Reliability engineering	Calculate and interpret applicable MTTF, MTBF, MTTR, availability, and observed failure rate. State formulas, units, observation windows, and assumptions.	Worked calculations using clearly identified experimental or explicitly illustrative data.
C. Architecture and infrastructure resilience	Show service boundaries, dependencies, data flows, redundancy/replication or backup strategy, and single points of failure. Demonstrate at least two infrastructure-level resilience mechanisms or justified simulations.	Architecture diagram, configuration, and failure demonstration.

D. Software fault tolerance	Implement at least four mechanisms, such as bounded retries/backoff, timeouts, circuit breaker, idempotency, duplicate-request detection, checkpoint/recovery, rollback, health checks, or graceful degradation.	Code/configuration references and tests showing each mechanism's behavior.
E. Cloud and microservices resilience	Define deployment and restart/replica behavior, health/readiness checks, dependency-failure behavior, and resource/load assumptions.	Deployment files or reproducible setup instructions and experiment results.
F. Monitoring and observability	Collect structured logs and at least three meaningful metrics; correlate requests; define at least three alert rules or incident indicators.	Dashboard/screenshots or exported data, log samples, and alert-test evidence.
G. Security and intrusion tolerance	Create a basic threat model and implement at least three applicable controls, including access control or authentication where relevant. Test at least two misuse/security scenarios in an authorized environment.	Threat model, control descriptions, test cases, and audit/denial evidence.
H. Verification, validation, and fault injection	Prepare acceptance criteria and automated or repeatable tests. Conduct at least eight experiments, including service/dependency failure, load, recovery/data integrity, observability, and security/misuse cases.	Test plan, scripts, raw results, pass/fail summary, and limitations.

5. Experimental Methodology

- Define a baseline configuration with resilience mechanisms disabled or minimized where safe and feasible.
- Define the improved configuration and document exactly which mechanisms differ from the baseline.
- Specify the hypothesis, workload, environment, duration, and success criteria before running each experiment.
- Repeat experiments where feasible and report the number of repetitions. Keep test conditions comparable.
- Record the injected fault or scenario, start/end times, detection time, recovery time, successful and failed requests, latency, and data-integrity result.
- Preserve raw logs/results and explain anomalies, confounding factors, and limitations.
- Do not invent measurements. If a value is simulated, estimated, or illustrative, label it clearly and do not present it as an observed result.

6. Minimum Experiment Matrix

No.	Scenario	Core question
1	Application/service instance failure	Does another instance or recovery mechanism keep the critical workflow available?
2	Database or dependency unavailable	Does the system fail safely, recover, and avoid corrupting data?
3	Network delay or timeout	Are timeout/retry policies bounded and effective?
4	Interrupted multi-step transaction	Can the system recover without losing or duplicating a critical record?
5	High load or traffic spike	How do throughput, latency, and errors change?
6	Monitoring and alert trigger	Can the incident be detected and diagnosed from available telemetry?
7	Unauthorized or invalid operation	Are access controls and validation enforced, and is the event logged?
8	Recovery/rollback or failover verification	Does the system return to an acceptable state and meet defined acceptance criteria?

7. Required Results and Analysis

Present a consolidated results table for all experiments and compare the baseline with the improved configuration. Include, where applicable:

- Availability or successful-request ratio over a defined interval
- Observed failure rate and the basis for the calculation
- MTTR/recovery time and incident-detection time
- Request success/failure counts and latency measurements
- Number of retries, circuit-breaker activations, failovers, or recovery actions
- Data consistency checks, including missing or duplicate critical records
- Security-test outcomes and evidence of expected denial or safe handling
- Trade-offs, such as added latency, complexity, resource use, or false-positive alerts

Distinguish measured values from theoretical estimates. Explain why results may differ from expected or theoretical behavior.

8. Final Report Structure

- Abstract and keywords
- Introduction, motivation, and system context
- Requirements, assumptions, and dependability objectives
- Fault model, threat model, and failure analysis
- Reliability engineering calculations
- System architecture and resilience design
- Infrastructure and software fault-tolerance implementation
- Monitoring, observability, and security implementation
- Verification, validation, and experimental methodology
- Results and baseline-versus-improved comparison
- Discussion, limitations, and future work
- Conclusion
- References
- Appendix: code repository, configuration, test scripts, logs, and supplementary tables

9. Deliverables

- Final technical report
- Source-code repository with README and reproducible setup instructions
- Architecture diagram and fault/threat models
- Baseline and improved configurations, or a clear description of how the comparison was performed
- Test suite and fault-injection scripts or documented repeatable procedures
- Raw experiment logs/data and consolidated results table
- Monitoring/observability evidence and security-test evidence
- Final presentation and live or recorded demonstration of at least four scenarios

10. Assessment Rubric

Criterion	Weight	Assessment focus
Requirements, fault model, and reliability analysis	15%	Clear objectives, realistic scenarios, correct calculations, and defensible assumptions.
Architecture and infrastructure resilience	15%	Sound architecture, redundancy/backup choices, and failure containment.
Software fault-tolerance implementation	20%	Correctness and demonstrated value of implemented mechanisms.

Observability and security	15%	Useful telemetry, incident detection, relevant security controls, and tested behavior.
Experiments, validation, and results	20%	Repeatability, fair comparisons, data integrity, measured evidence, and critical interpretation.
Report and reproducibility	10%	Technical clarity, complete documentation, references, and ability to reproduce results.
Presentation and demonstration	5%	Clear explanation, successful demonstration, and responses to technical questions.

11. Recommended Technology Stack

A suitable stack may include Python/FastAPI or another service framework, Docker, Kubernetes where appropriate, PostgreSQL or SQLite, Prometheus/Grafana or equivalent observability tools, and Git. Tool choice is flexible; teams must justify the stack and provide setup instructions that allow the instructor to inspect or reproduce the core experiments.

12. Academic and Technical Integrity

- Cite external libraries, datasets, code examples, and technical references.
- Clearly identify each team member's contribution.
- Do not include credentials, tokens, personal data, or confidential production logs in the repository or report.
- Report failed experiments and negative results honestly; a failed hypothesis is acceptable when analyzed rigorously.
- All demonstrations must run in a controlled environment with authorization.

13. Final Project Success Criteria

A successful final project provides a working, reproducible system; demonstrates multiple fault-tolerance and resilience mechanisms; evaluates at least eight controlled scenarios; presents defensible reliability and operational measurements; and explains whether the defined dependability objectives were met. Architectural claims without test evidence are insufficient.